

Kontraproduktívnosť princípov Clean Code a dopad na generatívnu AI

Marek Holik
Seminárna Práca | 2026

Univerzita Tomáša Baťu v Zlíne
FAI - Fakulta Aplikovanej Informatiky

Obsah

1	Úvod	5
2	Definícia Clean Code a súvisiacich pojmov.....	6
2.1	Kritika dogmatického uplatňovania Clean Code	6
2.2	Reálne príklady spojené s nadmerným uplatňovaním Clean Code.....	7
2.2.1	Nadmerná abstrakcia a architektonická zložitosť	7
2.2.2	Prehnané rozkladanie kódu	7
2.2.3	Nadmerná aplikácia návrhových vzorov a konfigurácie	7
2.2.4	Zbytočná architektonická a organizačná komplexita	7
3	Akým spôsobom AI generuje kód z repozitárov obsahujúci Clean Code?	9
3.1	Učenie zo zdrojového kódu a vzorov	9
3.2	Kontextové obohacovanie generovania	9
3.3	Riadenie výstupu pomocou inštrukcií	9
3.4	Odporúčania pri používaní AI pre generovanie Clean Code.....	10
3.5	Zťažuje AI a Clean Code testovateľnosť kódu?	10
3.5.1	Ako AI zlepšuje testovanie.....	10
3.5.2	Ako AI môže zhoršiť testovanie	10
3.6	Porovnanie ceny generovania kódu AI a nákladov na udržiavanie Clean Code.	11
3.6.1	AI Clean Code vs. AI Messy Code.....	11
4	Záver	12
	Referencie.....	13

1 Úvod

Princípy čistého kódu, ako napríklad malé funkcie, zmysluplné pomenovania, minimalizácia komentárov a princíp DRY (Don't Repeat Yourself), sa stali súčasťou štandardnej výučby programovania a sú často považované za správny prístup k vývoju softvéru vo veľkom množstve spoločností a boli široko prijaté agilnými tímami po celom svete [5].

Pôvodným cieľom Clean Code bolo zvýšiť čitateľnosť, udržiavateľnosť a kvalitu softvéru. V priebehu rokov sa však objavili názory, že niektoré odporúčania a postupy boli prijaté príliš rýchlo a nie sú nutne overené komunitou programátorov. Existujú názory, že snaha o dokonalý kód môže viesť k predčasnej optimalizácii, zbytočnému pridávaniu funkcionalít a nadmernému navrhovaniu systémov (over-engineeringu).

Tento problém je obzvlášť aktuálny v prostredí agilného vývoja, kde je prioritou rýchle dodávanie hodnoty používateľovi. Silná orientácia na architektonickú čistotu môže spomaľovať vývoj, zvyšovať komplexnosť systému a odvádzať pozornosť od skutočných potrieb používateľov. Výskumná otázka tejto práce teda znie:

Vedie dogmatické uplatňovanie princípov Clean Code k predčasnej optimalizácii, preťaženiu funkcionalitami a over-engineeringu?

Na zodpovedanie tejto otázky bude analyzovaných zopár názorov odborníkov z praxe, akademických zdrojov a súčasných diskusií o softvérovom inžinierstve. Cieľom je identifikovať situácie, v ktorých Clean Code prináša hodnotu, a situácie, v ktorých sa môže stať kontraproduktívnym.

2 Definícia Clean Code a súvisiacich pojmov

Na začiatok je potrebné zdefinovať pojem "Clean Code". V literatúre sa tento pojem často spája s knihou: **Clean Code: A Handbook of Agile Software Craftsmanship** od Roberta C. Martina [5], kde autor prezentuje súbor princípov, pravidiel a odporúčaní, ktoré majú za cieľ zlepšiť čitateľnosť, udržiavateľnosť a kvalitu písaného kódu. Medzi hlavné princípy Clean Code patrí:

- **Malé a zmysluplné funkcie:** Funkcie by mali byť krátke, robiť jednu vec a mať jasný názov, ktorý vystihuje ich účel.
- **Zmysluplné pomenovania:** Premenné, funkcie a triedy by mali mať názvy, ktoré jasne opisujú ich účel a zlepšujú čitateľnosť kódu.
- **Minimalizácia komentárov:** Kód by mal byť dostatočne čitateľný, aby nevyžadoval nadmerné komentovanie. Komentáre by mali byť použité len na vysvetlenie zložitých častí kódu alebo na poskytnutie kontextu.
- **Princíp DRY (Don't Repeat Yourself):** Opakujúci sa kód by mal byť refaktorovaný do samostatných funkcií alebo modulov, aby sa znížila duplicita a zvýšila udržiavateľnosť.

Téma Clean Code obsahuje aj súvisiace pojmy, ktoré sa často spomínajú v diskusiách o softvérovom inžinierstve a vývoji softvéru. Medzi tieto pojmy patrí:

Predčasná optimalizácia: Situácie, keď sa vývojári snažia optimalizovať kód predtým, než je to potrebné, čo môže viesť k zložitejšiemu a menej čitateľnému kódu. Predtým, než začneme optimalizovať, je dôležité mať jasnú predstavu o tom, kde sú skutočné problémy s výkonom a či je optimalizácia naozaj potrebná. Mikro-optimalizácie malých funkcií môžu mať minimálny dopad na celkový výkon komplexného systému [5].

Preťažovanie funkcionalitami: Pridávanie príliš veľa funkcií do systému, ktoré nie sú nevyhnutné pre jeho základnú funkcionality môže znižovať jeho udržiavateľnosť. S týmto súvisí aj pojem "feature creep", ktorý označuje postupné pridávanie nových funkcií do softvéru, často bez ohľadu na pôvodný plán alebo potreby používateľov. Taktiež z hľadiska Clean Code je spájaný tento pojem s DRY princípom, pretože pridávanie nadbytočných funkcií môže viesť k zníženiu čitateľnosti kódu [5].

Over-engineering: Nadmerné navrhovanie systémov s cieľom predvídať budúce potreby, čo môže viesť k zbytočne zložitým riešeniam. Aj keď Clean Code podporuje dobré návrhové princípy, nadmerné zameranie sa na architektonickú čistotu môže spôsobiť, že vývojári budú tráviť príliš veľa času navrhovaním riešení pre hypotetické scenáre [5].

2.1 Kritika dogmatického uplatňovania Clean Code

Niektorí autori a praktici upozorňujú, že princípy Clean Code by sa mali chápať ako usmernenie, nie ako pevne dané pravidlá. Clean Code zdôrazňuje, že čistota kódu je kontextuálna — čo je čitateľné a užitočné v jednom projekte, môže byť zbytočne komplikované v inom. Dôležité je hodnotiť prínos refaktoringu vzhľadom na dodávanú hodnotu a náklady na údržbu [3].

Konkrétne riziká dogmatického prístupu zahŕňajú zvýšenú kognitívnu záťaž, zbytočné vrstvy abstrakcie a skrytý technický dlh, ktorý vzniká pri pravidelnom prepracovávaní riešení bez jasného merania benefitov [4].

2.2 Reálne príklady spojené s nadmerným uplatňovaním Clean Code

Teraz sa môžeme pozrieť na konkrétne scenáre, ktoré signalizujú, že snaha písať čistý kód nemusí byť vždy najlepší nápad [4], [6]:

2.2.1 Nadmerná abstrakcia a architektonická zložitosť

Budovanie plugin systému pre funkciu s jediným spotrebiteľom: Pre jednoduchú funkcionálnu je vytvorená architektúra založená na pluginoch, registrácii modulov a dynamic-kom načítavaní implementácií, hoci existuje iba jeden spôsob použitia a nepredpokladá sa rozšírenie. Namiesto jednej triedy vzniká množstvo rozhraní, tovární a registračných mechanizmov. Výsledkom je vyššia zložitosť, náročnejšie ladenie a väčšie náklady na údržbu bez reálneho prínosu.

Generalizácia pred prvou potrebou: Vývojári vytvoria všeobecný framework alebo konfigurovateľný systém ešte predtým, než sa objaví druhý prípad použitia. Predpokladané rozšírenia sa často nikdy nezrealizujú, zatiaľ čo vytvorená infraštruktúra zostáva súčasťou projektu a zvyšuje jeho komplexitu. Na jednej strane sa snažíme o flexibilitu a dá sa to brať ako investícia do budúcnosti, no na druhej strane sa môže stať, že táto flexibilita nikdy nebude využitá a stane sa zbytočnou záťažou [4], [6].

2.2.2 Prehnané rozkladanie kódu

Rozdeľovanie logiky do extrémne malých metód: Snaha o dodržanie pravidla „jedna funkcia robí jednu vec“ vedie k vytváraniu metód obsahujúcich jeden alebo dva riadky kódu. Programátor je nútený neustále preskakovať medzi definíciami funkcií, čo znižuje čitateľnosť a predlžuje ladenie.

2.2.3 Nadmerná aplikácia návrhových vzorov a konfigurácie

Používanie návrhových vzorov bez reálnej potreby: Pre jednoduché vytvorenie objektu sú implementované Factory, Builder alebo Strategy vzory, hoci postačuje priama konštrukcia objektu. Návrhové vzory sa stávajú cieľom samy osebe namiesto riešenia konkrétneho problému.

Konfigurovateľnosť všetkého: Systém umožňuje konfigurovať správanie každej komponenty pomocou externých súborov, databázových nastavení alebo dependency injection kontajnerov. V praxi sa väčšina konfigurácií nikdy nemení, no aplikácia obsahuje množstvo dodatočného kódu potrebného na ich správu [4], [6].

2.2.4 Zbytočná architektonická a organizačná komplexita

Abstrakcia nad stabilnými technológiami: Nad databázou, súborovým systémom alebo knižnicou je vytvorená vlastná abstrakčná vrstva pre prípad budúcej zmeny technológie. V mnohých projektoch však k takejto zmene nikdy nedôjde. Vývojári tak musia dlhodobo udržiavať dodatočnú vrstvu bez reálneho využitia.

Refaktorovanie fungujúceho kódu bez obchodnej hodnoty: Významná časť času je venovaná prepisovaniu existujúceho kódu s cieľom dosiahnuť čistejšiu architektúru. Funkcionalita sa nemení, používateľ nezískava nové možnosti a organizácia nenadobúda žiadnu priamu hodnotu, zatiaľ čo rastie riziko zavedenia nových chýb.

Mikroslužby pre malý systém: Aplikácia s niekoľkými obrazovkami a jednou databázou je rozdelená na viacero mikroslužieb kvôli architektonickej čistote. Namiesto jednoduchého nasadenia vznikajú problémy s komunikáciou služieb, monitoringom, logovaním a orchestráciou [4], [6].

Autori, ktorí skúmajú skryté náklady Clean Code odporúčajú merať dopad takýchto rozhodnutí a preferovať pragmatické riešenia, kým sa nepreukáže potreba zložitosti [4].

Prakticky je odporúčané najprv merať (profilovanie, telemetria, používateľská spätná väzba), až potom optimalizovať alebo refaktorovať. Zamerať sa na rozhrania, kontrakty a testovateľné správanie často prináša výraznejší benefit [3], [4].

3 Akým spôsobom AI generuje kód z repozitárov obsahujúci Clean Code?

Generovanie kódu pomocou modelov umelej inteligencie kombinuje viaceré techniky a zdroje informácií. Pri práci s repozitármi, ktoré obsahujú príklady dodržiavajúce princípy Clean Code, možno identifikovať niekoľko hlavných mechanizmov, ktoré ovplyvňujú výsledný štýl a kvalitu generovaného kódu:

3.1 Učenie zo zdrojového kódu a vzorov

Tréningové dáta a vzory: Modely sa učia štatistické vzory z veľkých množín zdrojových kódov. Ak sú v tréningovej množine zastúpené repozitáre dodržiavajúce Clean Code (čisté pomenovania, krátke funkcie, jednotné konvencie), model sa naučí preferovať podobné štruktúry a šablóny pri generovaní.

Retrieval-augmented generation: Pri generovaní špecifického úryvku kódu môže systém využiť dohľadávacie mechanizmy, ktoré do kontextu priradia relevantné súbory alebo útržky z repozitára (README, existujúce implementácie, testy). Tým sa zvyšuje konzistencia s miestnymi konvenciami a štýlom.

3.2 Kontextové obohacovanie generovania

Signály kvality v repozitári: Ak repozitár obsahuje testy, konfiguračné súbory, CI skripty a komentáre, tieto prvky slúžia ako signály o preferovanom štýle a očakávanom správaní. Modely a nástroje postavené nad nimi môžu tieto signály použiť na generovanie kódu, ktorý spĺňa lokálne pravidlá (napr. štýl, ošetrovanie chýb).

Fine-tuning a špecializované modely: Pre konkrétne organizácie alebo projekty sa často modely doladujú na interných repozitároch a dokumentáciach, čím sa ešte viac priblížia k miestnym praktikám Clean Code.

3.3 Riadenie výstupu pomocou inštrukcií

Prompt engineering a kontextové inštrukcie: Aby sme zaručili správny typ generovaného textu, priame inštrukcie (napr. "dodržuj princípy Clean Code", "vytvor testovateľnú funkciu s jasným názvom") v prompt-e výrazne ovplyvňujú kvalitu výstupu. Dobré prompty spolu s relevantným kontextom z repozitára vedú k kvalitnejším a konzistentnejším odpovediam. Napriek tomu existujú obmedzenia a riziká:

- **Nedostatok celkového kontextu:** Model môže generovať lokálne korektný kód, ktorý však neberie do úvahy širšie architektonické rozhodnutia alebo implicitné zmluvy rozhrania.
- **Licenčné a etické otázky:** Používanie verejných repozitárov do tréningu alebo pri retrieval môže viesť k problémom s licenciami a atribúciou. Niektoré modely môžu byť trénované na veľkých kolekciami verejne dostupného zdrojového kódu, čo vyvoláva otázky týkajúce sa licencií a atribúcie, a preto teda generovaný kód obsahuje fragmenty chránené autorskými právami.
- **Falošná istota v syntaxi:** AI môže vytvoriť kód, ktorý vyzerá čisto no obsahuje logické

chyby alebo neefektívne riešenia.

3.4 Odporúčania pri používaní AI pre generovanie Clean Code

Pri nasadzovaní AI do vývojového procesu je odporúčané dbať na tieto zásady [2]:

- **Automatická kontrola:** Je vhodné integrovať generovanie s testami tak, aby sa každý návrh overil pred prijatím.
- **Konzistencia kontextu:** Na zachovanie konzistencie sa používa metóda retrieval z lokálnych repozitárov a zahrňuje štýlové a konvenčné súbory (README, konfiguračné súbory), aby bol výstup v súlade s projektovými pravidlami.
- **Testy spolu s implementáciou:** Vyžadujeme od AI generovanie súvisiacich unit a integration testov a príkladov použitia spolu s implementáciou.
- **Ľudský dohľad:** Je dôležité nezlenivieť a robiť striktné code reviews a posudzovať návrhy ľudským recenzentom. Výstupy AI sa musia vždy považovať za návrhy, nie za finálne riešenia.

3.5 Zľahuje AI a Clean Code testovateľnosť kódu?

Vplyv AI na testovateľnosť kódu nie je jednoznačný. Môže ju buď zlepšiť, alebo zhoršiť, v závislosti od spôsobu použitia [2].

3.5.1 Ako AI zlepšuje testovanie

Generovanie testov: AI dokáže na základe špecifikácie alebo existujúcej implementácie navrhnúť jednotkové testy, integračné testy alebo príklady použitia, čo zrýchľuje pokrytie testami.

Podpora TDD a test-first promptov: Pri zadaní "najprv napíš testy, potom implementáciu" môže AI vygenerovať konzistentnú dvojicu test + implementácia, čím sa podporuje testovateľná architektúra.

Refaktoring pre testovateľnosť: AI môže navrhnúť rozdelenie veľkých metód, zavedenie závislostí cez dependency injections alebo izolovanie vedľajších efektov — všetko v prospech testovateľnosti.

3.5.2 Ako AI môže zhoršiť testovanie

Povrchne správny, hlboko nesprávny kód: Generovaný kód môže prejsť jednoduchými testami, no byť krehký pri okrajových prípadoch. Ak testy nie sú komplexné, poskytujú falošný pocit správnej implementácie.

Chýbajúca doménová znalosť: Bez hlbšieho porozumenia domény AI môže vytvoriť rozhrania alebo vedľajšie efekty, ktoré sú ťažko testovateľné.

Abstrakcia: AI môže automaticky zaviesť abstrakcie, ktoré zjednodušia kód na povrchu, ale skomplikujú písanie izolovaných testov, alebo naopak vytvoriť monolitické metódy, ktoré sú ťažké na unit testovanie.

Celkovo je dôležité vnímať AI ako nástroj, ktorý môže zrýchliť vytváranie testovateľného a Clean Code, no bez dobre nastavených pravidiel, metrik a ľudského dohľadu môže viesť k zanedbaniu kvality. Najlepšie výsledky sa dosahujú kombináciou automatizovaných nástrojov a jasne definovaných procesov.

3.6 Porovnanie ceny generovania kódu AI a nákladov na udržiavanie Clean Code

Posledná otázka, ktorej sa v práci budeme venovať, je porovnanie nákladov na generovanie kódu pomocou AI a nákladov na udržiavanie Clean Code. Pozrime sa na túto tému z hľadiska času. AI sama o sebe umožňuje zrýchliť vývoj softvéru za cenu, ktorú by developer ušeril namiesto písania kódu, teda môžeme považovať tieto ceny za ekvivalentné a samotné použitie AI automaticky negarantuje zníženie ani zvýšenie nákladov na vývoj softvéru. Rozhodujúce je, či sa generovaný kód integruje, prejde kvalitným review a je dostatočne pochopený tímom. Keďže moderné modely generujú väčšinou syntakticky správny kód, ide hlavne o správnu architektúru [1].

3.6.1 AI Clean Code vs. AI Messy Code

Pri **AI Clean Code** sa technický dlh zvyšuje štandardným tempom približne o 10–15 % za 90 dní. Náklady na údržbu zostávajú v prvom roku na úrovni bežného vývoja, v druhom roku rastú na približne 1,2–1,5-násobok a v dlhodobom horizonte dosahujú približne 1,5-násobok pôvodných nákladov. Miera výskytu "code smells" je približne 45 %, čo zodpovedá úrovni ľudsky písaného kódu. Náklady na opravu chýb, čas potrebný na revíziu kódu aj náročnosť testovania zostávajú na štandardnej úrovni [1].

Pri **AI Messy Code** rastie technický dlh výrazne rýchlejšie, približne o 30–41 % za 90 dní. Hoci sú náklady na údržbu počas prvého roka podobné ako pri čistom kóde, v druhom roku dosahujú približne 2,5-násobok a po troch a viac rokoch až 4-násobok pôvodnej hodnoty. Miera výskytu "code smells" sa pohybuje medzi 78 a 92 % v závislosti od použitého nástroja. Oprava chýb je 3–4-krát drahšia v dôsledku horšieho porozumenia kontextu kódu, proces "code review" si vyžaduje približne o 9 % viac času a testovanie odhaľuje približne 1,7-krát viac defektov [1].

4 Záver

Cieľom tejto práce bolo analyzovať princípy Clean Code, ich praktické využitie a vzťah k moderným nástrojom umelej inteligencie používaným pri vývoji softvéru. Ukázalo sa, že Clean Code predstavuje súbor odporúčaní zameraných na zlepšenie čitateľnosti, testovateľnosti a udržiavateľnosti kódu, avšak jeho aplikácia by nemala byť dogmatická. Nadmerná abstrakcia, predčasná optimalizácia alebo zbytočná architektonická komplexita môžu viesť k vyšším nákladom na vývoj a údržbu bez primeraného prínosu.

Práca zároveň poukázala na to, že nástroje umelej inteligencie dokážu významne urýchliť tvorbu zdrojového kódu, generovanie testov a refaktoring existujúcich riešení. Kvalita výsledného kódu však nezávisí iba od schopností modelu, ale predovšetkým od kvality vstupného kontextu, spôsobu integrácie do vývojového procesu a úrovne ľudského dohľadu. AI je schopná generovať kód, ktorý spĺňa mnohé princípy Clean Code, no zároveň môže vytvárať riešenia obsahujúce architektonické nedostatky, technický dlh alebo logické chyby.

Porovnanie čistého a nekonzistentného AI generovaného kódu ukazuje, že krátkodobé zrýchlenie vývoja nemusí automaticky znamenať dlhodobú úsporu nákladov. Zatiaľ čo kvalitne kontrolovaný AI generovaný kód dosahuje podobné náklady na údržbu ako tradičný vývoj, nekvalitný kód môže v horizonte niekoľkých rokov výrazne zvýšiť technický dlh a náklady na údržbu systému.

Na základe získaných poznatkov možno konštatovať, že najefektívnejším prístupom je kombinácia princípov Clean Code, automatizovaných kontrol kvality, testovania a pravidelného code review. Umelá inteligencia by mala byť vnímaná ako podporný nástroj, ktorý zvyšuje produktivitu vývojárov, nie ako náhrada za odborné rozhodovanie. Dlhodobú kvalitu softvéru zabezpečuje predovšetkým schopnosť tímu rozumieť vytvorenému kódu, priebežne merať jeho kvalitu a robiť pragmatické rozhodnutia založené na reálnych potrebách projektu.

Referencie

- [1] **cto_michelle**. „75% of Tech Leaders Will Face Moderate or Severe AI Technical Debt by 2026—Yet We’re Still Measuring Productivity by Lines of Code. What Should We Track Instead? “URL: <https://tianpan.co/forum/t/75-of-tech-leaders-will-face-moderate-or-severe-ai-technical-debt-by-2026-yet-were-still-measuring-productivity-by-lines-of-code-what-should-we-track-instead/4287>
- [2] **freeCodeCamp**. „How AI Changed the Economics of Writing Clean Code. “URL: <https://www.freecodecamp.org/news/how-ai-changed-the-economics-of-writing-clean-code/>
- [3] **S. Hughes**. „Clean. “URL: <https://qntm.org/clean>
- [4] **O. Keswani**. „The Hidden Cost of “Clean Code”: When Over-Engineering Quietly Kills Shipping Velocity. “URL: https://dev.to/omieee_24/the-hidden-cost-of-clean-code-when-over-engineering-quietly-kills-shipping-velocity-1pg7
- [5] **R. C. Martin**, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008, ISBN: 978-0-13-235088-4. URL: <https://www.lkhibra.ma/books/clean-code.pdf>
- [6] **Reddit r/AskProgramming**. „Why do some people hate “Clean Code”?“ Discussion thread on Reddit. URL: https://www.reddit.com/r/AskProgramming/comments/1j3p9bi/why_do_some_people_hate_clean_code/